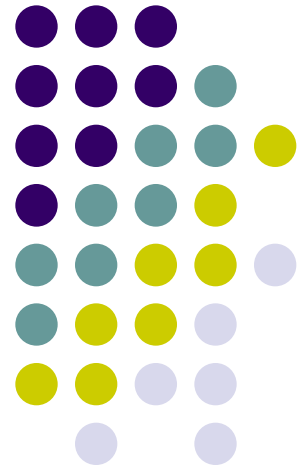


php



Teacher: Wafaa Alnbhany

Lecture13



PHP Error Handling

The default error handling in PHP is very simple. An error message with filename, line number and a message describing the error is sent to the browser.

PHP Error Handling

When creating scripts and web applications, error handling is an important part. If your code lacks error checking code, your program may look very unprofessional and you may be open to security risks.

This tutorial contains some of the most common error checking methods in PHP.

We will show different error handling methods:

- Simple "die()" statements
- Custom errors and error triggers
- Error reporting

Basic Error Handling: Using the die() function

The first example shows a simple script that opens a text file:

```
<?php
$file=fopen("welcome.txt","r");
?>
```

If the file does not exist you might get an error like this:

```
Warning: fopen(welcome.txt) [function.fopen]: failed to open
stream:
No such file or directory in C:\webfolder\test.php on line 2
```

To avoid that the user gets an error message like the one above, we test if the file exist before we try to access it:

```
<?php
if(!file_exists("welcome.txt"))
{
    die("File not found");
}
else
{
    $file=fopen("welcome.txt","r");
}
?>
```

Now if the file does not exist you get an error like this:

```
File not found
```

The code above is more efficient than the earlier code, because it uses a simple error handling mechanism to stop the script after the error.

However, simply stopping the script is not always the right way to go. Let's take a look at alternative PHP functions for handling errors.

Creating a Custom Error Handler

Creating a custom error handler is quite simple. We simply create a special function that can be called when an error occurs in PHP.

This function must be able to handle a minimum of two parameters (error level and error message) but can accept up to five parameters (optionally: file, line-number, and the error context):

Syntax

```
error_function(error_level,error_message,  
error_file,error_line,error_context)
```

Parameter	Description
error_level	Required. Specifies the error report level for the user-defined error. Must be a value number. See table below for possible error report levels
error_message	Required. Specifies the error message for the user-defined error
error_file	Optional. Specifies the filename in which the error occurred
error_line	Optional. Specifies the line number in which the error occurred
error_context	Optional. Specifies an array containing every variable, and their values, in use when the error occurred

Error Report levels

These error report levels are the different types of error the user-defined error handler can be used for:



Value	Constant	Description
2	E_WARNING	Non-fatal run-time errors. Execution of the script is not halted
8	E_NOTICE	Run-time notices. The script found something that might be an error, but could also happen when running a script normally
256	E_USER_ERROR	Fatal user-generated error. This is like an E_ERROR set by the programmer using the PHP function <code>trigger_error()</code>
512	E_USER_WARNING	Non-fatal user-generated warning. This is like an E_WARNING set by the programmer using the PHP function <code>trigger_error()</code>
1024	E_USER_NOTICE	User-generated notice. This is like an E_NOTICE set by the programmer using the PHP function <code>trigger_error()</code>
4096	E_RECOVERABLE_ERROR	Catchable fatal error. This is like an E_ERROR but can be caught by a user defined handle (see also <code>set_error_handler()</code>)
8191	E_ALL	All errors and warnings, except level E_STRICT (E_STRICT will be part of E_ALL as of PHP 6.0)

Now lets create a function to handle errors:

```
function customError($errno, $errstr)
{
echo "<b>Error:</b> [$errno] $errstr<br />";
echo "Ending Script";
die();
}
```

The code above is a simple error handling function. When it is triggered, it gets the error level and an error message. It then outputs the error level and message and terminates the script.

Now that we have created an error handling function we need to decide when it should be triggered.

Set Error Handler

The default error handler for PHP is the built in error handler. We are going to make the function above the default error handler for the duration of the script.

It is possible to change the error handler to apply for only some errors, that way the script can handle different errors in different ways. However, in this example we are going to use our custom error handler for all errors:

```
set_error_handler("customError");
```

Since we want our custom function to handle all errors, the `set_error_handler()` only needed one parameter, a second parameter could be added to specify an error level.

Example

Testing the error handler by trying to output variable that does not exist:

```
<?php
//error handler function
function customError($errno, $errstr)
{
    echo "<b>Error:</b> [$errno] $errstr";
}
//set error handler
set_error_handler("customError");
//trigger error
echo($test);
?>
```

The output of the code above should be something like this:

```
Error: [8] Undefined variable: test
```

Trigger an Error

In a script where users can input data it is useful to trigger errors when an illegal input occurs. In PHP, this is done by the `trigger_error()` function.

Example

In this example an error occurs if the "test" variable is bigger than "1":

```
<?php
$test=2;
if ($test>1)
{
    trigger_error("Value must be 1 or below");
}
?>
```

The output of the code above should be something like this:

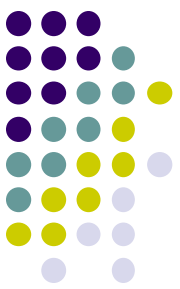
```
Notice: Value must be 1 or below
in C:\webfolder\test.php on line 6
```

An error can be triggered anywhere you wish in a script, and by adding a second parameter, you can specify what error level is triggered.

Possible error types:

- `E_USER_ERROR` - Fatal user-generated run-time error. Errors that can not be recovered from. Execution of the script is halted
- `E_USER_WARNING` - Non-fatal user-generated run-time warning. Execution of the script is not halted
- `E_USER_NOTICE` - Default. User-generated run-time notice. The script found something that might be an error, but could also happen when running a script normally

Example



In this example an E_USER_WARNING occurs if the "test" variable is bigger than "1". If an E_USER_WARNING occurs we will use our custom error handler and end the script:

```
<?php
//error handler function
function customError($errno, $errstr)
{
    echo "<b>Error:</b> [$errno] $errstr<br />";
    echo "Ending Script";
    die();
}
//set error handler
set_error_handler("customError",E_USER_WARNING);
//trigger error

$test=2;
if ($test>1)
{
    trigger_error("Value must be 1 or below",E_USER_WARNING);
}
?>
```

The output of the code above should be something like this:

```
Error: [512] Value must be 1 or below
Ending Script
```

Now that we have learned to create our own errors and how to trigger them, lets take a look at error logging.

Error Logging

By default, PHP sends an error log to the servers logging system or a file, depending on how the `error_log` configuration is set in the `php.ini` file. By using the `error_log()` function you can send error logs to a specified file or a remote destination.

Sending errors messages to yourself by e-mail can be a good way of getting notified of specific errors.

Send an Error Message by E-Mail

In the example below we will send an e-mail with an error message and end the script, if a specific error occurs:

```
<?php
//error handler function
function customError($errno, $errstr)
{
    echo "<b>Error:</b> [$errno] $errstr<br />";
    echo "Webmaster has been notified";
    error_log("Error: [$errno] $errstr",1,
        "someone@example.com","From: webmaster@example.com");
}
//set error handler
set_error_handler("customError",E_USER_WARNING);
//trigger error
$test=2;
if ($test>1)
```



```
{  
trigger_error("Value must be 1 or below",E_USER_WARNING);  
}  
?>
```

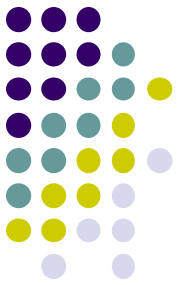
The output of the code above should be something like this:

```
Error: [512] Value must be 1 or below  
Webmaster has been notified
```

And the mail received from the code above looks like this:

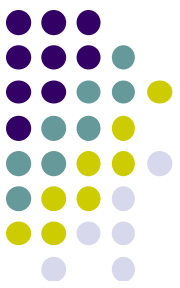
```
Error: [512] Value must be 1 or below
```

This should not be used with all errors. Regular errors should be logged on the server using the default PHP logging system.



error_log() log types

0	message is sent to PHP's system logger, using the Operating System's system logging mechanism or a file, depending on what the error log configuration directive is set to. This is the default option.
1	message is sent by email to the address in the destination parameter. This is the only message type where the fourth parameter, additional_headers is used.
2	No longer an option.
3	message is appended to the file destination.



Validation form

firstname: lastname:

Username: password: rewrite password

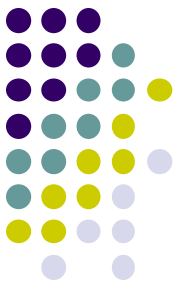
City: Zip:

Product Detials: email

phone gender

personal photo No file chosen

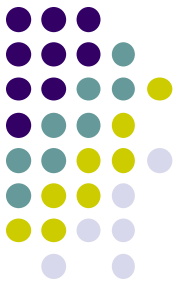
☐ Agree to terms and conditions



users Table

	Field	Type
☐	Firstname	varchar(20)
☐	Lastname	varchar(20)
☐	Username	varchar(20)
☐	password	varchar(20)
☐	City	varchar(20)
☐	Zip	varchar(20)
☐	Productdetials	varchar(20)
☐	email	varchar(100)
☐	phone	int(12)
☐	gender	int(2)
☐	img_name	varchar(20)
☐	img_type	varchar(20)
☐	img_path	varchar(100)

Input form



firstname: lastname:

Username: password: rewrite password

City: Zip:

Product Detials: email

phone gender

personal photo No file chosen

☐ Agree to terms and conditions

```
<form action="" method="post" enctype="multipart/form-data">
  <tr>
    <td>firstname:</td>
    <td><input type="text" name="fir"></td>
  </tr>
  <tr>
    <td>lastname:</td>
    <td><input type="text" name="last"></td>
  </tr>
  <tr>
    <td>Username:</td>
    <td><input type="text" name="User" require></td>
  </tr>
  <tr>
    <td>password:</td>
    <td><input type="password" name="pass" require></td>
  </tr>
  <tr>
    <td>rewrite password</td>
    <td><input type="password" name="pass2" require></td>
  </tr>
  <tr>
    <td>City:</td>
    <td><input type="text" name="City" require></td>
  </tr>
  <tr>
    <td>Zip:</td>
    <td><input type="text" name="Zip" require></td>
  </tr>
  <tr>
    <td>Product Detials:</td>
    <td><input type="text" name="pdetial" require></td>
  </tr>
  <tr>
    <td>email</td>
    <td><input type="text" name="email" require></td>
  </tr>
  <tr>
    <td>phone</td>
    <td><input type="text" name="phone" require></td>
  </tr>
  <tr>
    <td>gender</td>
    <td><input type="text" name="gender" require></td>
  </tr>
  <tr>
    <td>personal photo</td>
    <td><input type="file" name="file" id="file" /></td>
  </tr>
  <tr>
    <td><input type="checkbox" value="" required /></td>
    <td><label>Agree to terms and conditions</label></td>
  </tr>
  <tr>
    <td colspan="2"><input type="submit" value="submit" name="ok"></td>
  </tr>
</form>
```



```
<?php
include_once 'db.php';
$E_user_name= "";
$E_pass1     = "";
$E_pass2     = "";
if (isset($_POST['ok'])) {
    $f = $_POST["fir"];
    $l = $_POST["last"];
    $u = $_POST["User"];
    $p = strval($_POST["pass"]);
    $p2= strval($_POST["pass2"]);
    $c = $_POST["City"];
    $z = $_POST["Zip"];
    $pd = $_POST["pdetial"];
    $email= $_POST["email"];
    $phone= $_POST["phone"];

    // strong password
    $uppercas = preg_match('@[A-Z]@', $p);
    $lowercas = preg_match('@[a-z]@', $p);
    $specialchars = preg_match('@[^\w]@', $p);
    $number = preg_match('@[0-9]@', $p);

    if (!(ctype_alpha($f))) {
        $E_user_name = "you must enter the name or enter only alphabetic characters";
        echo $E_user_name ; }
    else if ($p != $p2) {
        $E_pass2 = "Please rewrite match password";
        echo $E_pass2;
    }
}
```

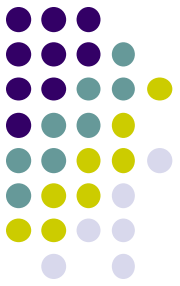


```
- else if (empty($p) || !$supercas || !$lowercas || !$specialchars || strlen($p) < 4){
$E_pass1= "password should be at least 4 characters in length and should include
at least one number, one upper case letter, one lowerc case letter and \w characters";
-echo $E_pass1; }

else if (empty($email) || (!(filter_var($email,FILTER_VALIDATE_EMAIL))))
{echo "your email either empty or invalid";}

-else {
$folder = "/upload";
$img= $_FILES["file"]["name"];
$tmp= $_FILES["file"]["tmp_name"];
$type= $_FILES["file"]["type"];
$path= "upload/" . $_FILES["file"]["name"];
$sql = "INSERT INTO test4.user (Firstname,Lastname,Username,password,City ,Zip
, Productdetials ,phone, email, img_name, img_type, img_path)
VALUES ('$f','$l','$u','$p','$c','$z','$pd','$phone', '$email', '$img', '$type','$path' )";
-if (mysqli_query($conn, $sql)) {
echo "New record has been added successfully !";
} else {
echo "Error: " . $sql . ":-" . mysqli_error($conn);
}
move_uploaded_file($_FILES["file"]["tmp_name"],
"upload/" . $_FILES["file"]["name"]);
echo "file Stored in: " . "upload/" . $_FILES["file"]["name"];
mysqli_close($conn);
}}
?>
```

H.W



- Create a website, the duration of the session is only one hour, and then the site is closed
 - Create custom error handler that present like the follow (your name is the function name)
- The error: [8] undefined variable : please check the code